



Виртуализация

Виртуализация – «отделение» среды исполнения ПО от физической среды:

1. Среда исполнения ограниченного набора программных компонент одной ОС в другой (CygWin, MinGV)
2. Виртуальная среда исполнения кода. Java (JVM)
3. Виртуальная ОС
4. Контейнер – создание собственной среды исполнения приложения, отдельного от других приложений ОС (зависимости от версий библиотек, СУБД, настройки сети и т.п.)

Изучаем подсистемы Windows для Linux, (перевод) Прэйттик Сингх

<http://onreader.mdl.ru/LearnWindowsSubsystemLinuxProfessionals/content/index.html>

Контейнеры Linux и виртуализация (перевод) Шашанка Мохана Джейна

<http://onreader.mdl.ru/LinuxContainersVirtualizationKernelPerspective/content/Ch01.html>



1. Виртуализация на уровне API. CygWin

Cygwin - это Linux-подобная среда для систем на базе Windows. Он состоит из эмулятора и набора инструментов, которые обеспечивают возможность работы на **Linux** в среде **Windows**.

Cygwin состоит из DLL `cygwin1.dll`, которая действует как уровень эмуляции, обеспечивающий функциональность системного вызова POSIX через Windows. С Cygwin пользователи имеют доступ к стандартным утилитам UNIX, которые могут использоваться либо из предоставленной `bash`-оболочки, либо через командную строку Windows.

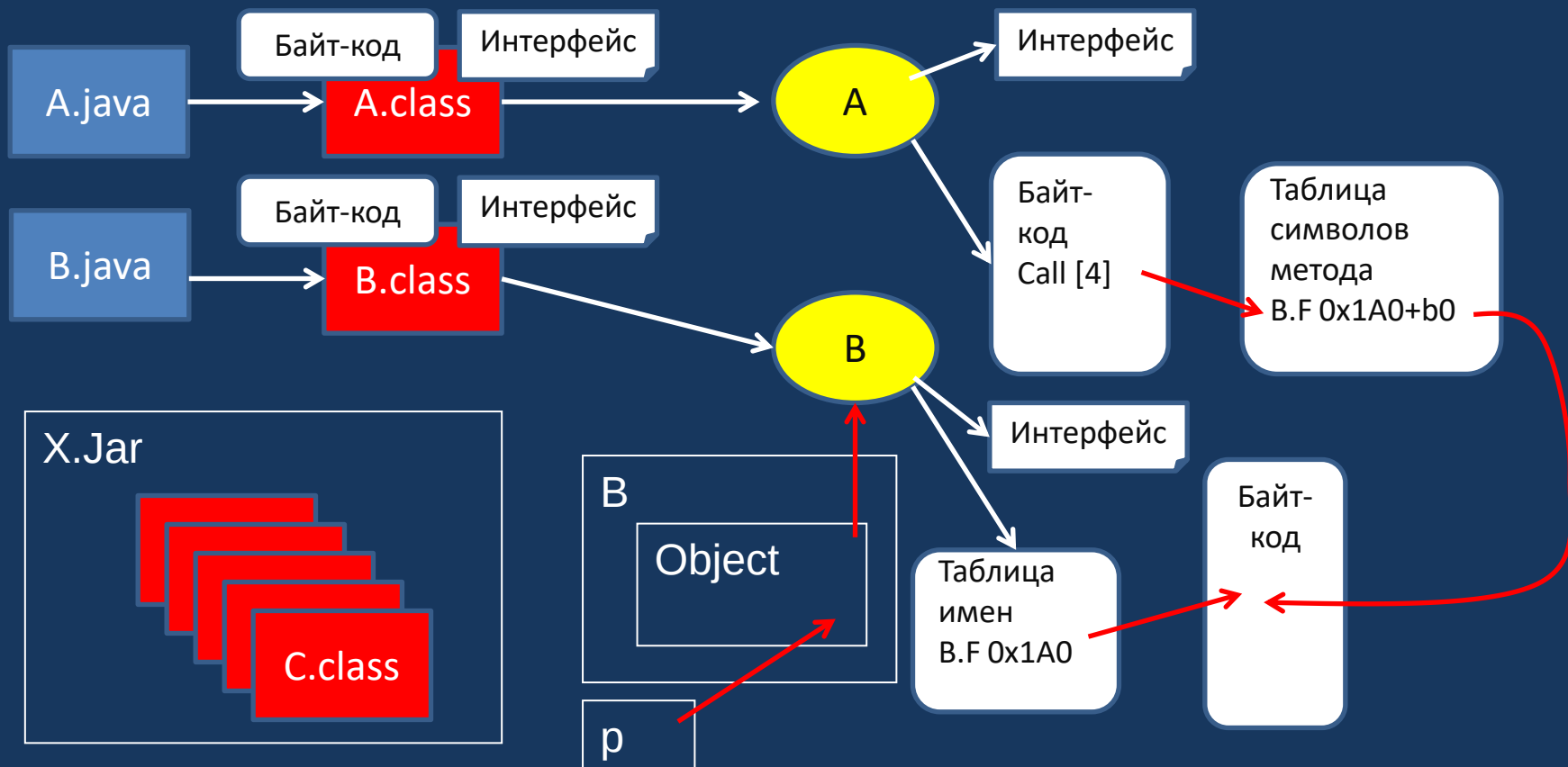
Cygwin - это порт утилит Gnu под Windows, а **Gnu** (www.gnu.org) - это проект Фонда Свободного Программного Обеспечения (Free Software Foundation, или просто FSF), ставящий своей целью создание некоммерческой Unix системы, не принадлежащей никому в отдельности и свободной от ограничивающих свободу распространения и модификации ПО лицензий:

- `gcc` - Gnu C Compiler, или Gnu Compiler Collection, коллекция компиляторов
- `bash` - командный интерпретатор Bourne Again Shell
- Emacs - семейство многофункциональных расширяемых текстовых редакторов

Cygwin - Эта прокладка `cygwinx.dll` (x - номер версии) - обеспечивает эмуляцию системных вызовов UNIX, что позволяет компилировать и исполнять Unix программы без или почти без изменения исходного кода. В принципе, эта dll и есть сам Cygwin, а все остальное - программные пакеты GNU, *скомпилированные* для работы с Cygwin



2. Виртуальная среда исполнения кода. Java





Виртуальная среда исполнения кода. Java

Динамическая объектно-ориентированная модель представления программы:

- Java-программа представляет собой набор классов, для каждого класса при компиляции создается двоичный файл с расширением **class**, содержащий кроме собственно *байт-кода методов* еще и *описание структуры класса (описание данных и заголовки методов)*
- JVM при загрузке классов для каждого из них создает объект класса **Class** – описатель класса, в который загружаются данные из файла описания
- Описатель класса содержит ссылки на родительский класс, интерфейсы и т.п.

Динамическое связывание внешних методов и данных:

- Вызов внешнего метода «по имени»
- При вызове методов в «посторонних» классах байт-код содержит ссылку на таблицу символов метода с именем внешней ссылки
- Ищется описатель класса **Class**, в его таблице символов ищется имя и адрес точки входа в байт-коде класса

JNI – Java Native Interface – переход к платформенно зависимому коду (Си++) при помощи вызова native-методов, которые получают параметры окружения JVM и набор сервисных функций



Java. Структуры данных JVM

Набор регистров JVM:

- **pc** - регистр-указатель на команду;
- **optop** - регистр-указатель на вершину стека операндов текущего кадра;
- **var** - регистр-указатель на массив локальных переменных текущего кадра;
- **frame** - регистр-указатель на среду выполнения текущего метода.

Стек потока – список кадров (**frame**) вызванных методов.

Структура кадра:

- массив локальных переменных объекта, **var** – начальный адрес (размерность при компиляции, операнды – индексы в массиве);
- стек операндов, **optop** – верхушка стека;
- структуры среды выполнения, **frame** - указатель.





Среда исполнения метода

Среда выполнения метода содержит информацию, необходимую для **динамического связывания**, возврата из метода и обработки исключений. Код класса (размещенный в области класса) обращается к внешним методам и переменным, используя символические ссылки. Динамическая компоновка переводит символические ссылки в фактические. Среда выполнения содержит ссылки **на таблицу символов метода**, через которую производятся обращения к внешним методам и переменным.

Данные для возврата из метода:

- указатель на кадр вызывающего метода
- pc – точка возврата
- содержимое регистров вызывающего метода
- указатель на область для записи возвращаемого значения

Ссылки на секции обработки исключений в методе класса

Через среду выполнения также происходят обращения к данным, содержащимся в области класса, в том числе, к константам и к переменным класса.



Система команд JVM

Команды загрузки и сохранения:

- загрузка в стек локальной переменной;
- сохранение значения из стека в локальной переменной;
- загрузка в стек константы (из пула констант).

Команды манипулирования значениями (**стековые**):

- арифметические операции
- побитовые логические операции, сдвиг
- инкремент (операция работает с операндом - локальной переменной).

Команды преобразования типов

Команды создания ссылочных данных и доступа к ним:

- создания экземпляров класса
- доступа к полям класса
- создания массивов, чтения в стек и сохранения элементов массивов;
- получения свойств массивов и объектов

Команды прямого манипулирования со стеком

Команды передачи управления:

- безусловный переход
- условный переход
- переход по множественному выбору

Команды вызова методов и возврата

Команды генерации и обработки исключений



Java. «Реассемблированный» байт-код

```
public class StatefulWriter extends WriterWrapper {
```

```
    public static int STATE_OPEN;  
    public static int STATE_NODE_START;  
    public static int STATE_VALUE;  
    public static int STATE_NODE_END;  
    public static int STATE_CLOSED;  
    private transient int state;  
    private transient int balance;  
    private transient FastStack attributes;
```

```
    public StatefulWriter(HierarchicalStreamWriter wrapped) {
```

```
        // <editor-fold defaultstate="collapsed" desc="Compiled Code">
```

```
        /* 0: aload_0
```

```
         * 1: aload_1
```

```
         * 2: invokespecial com/thoughtworks/xstream/io/WriterWrapper."<init>": (Lcom/thou
```

```
         * 5: aload_0
```

```
         * 6: getstatic      com/thoughtworks/xstream/io/StatefulWriter.STATE_OPEN:I
```

```
         * 9: putfield      com/thoughtworks/xstream/io/StatefulWriter.state:I
```

```
         * 12: aload_0
```

```
         * 13: new          com/thoughtworks/xstream/core/util/FastStack
```

```
         * 16: dup
```

```
         * 17: bipush      16
```

```
         * 19: invokespecial com/thoughtworks/xstream/core/util/FastStack."<init>": (I)V
```

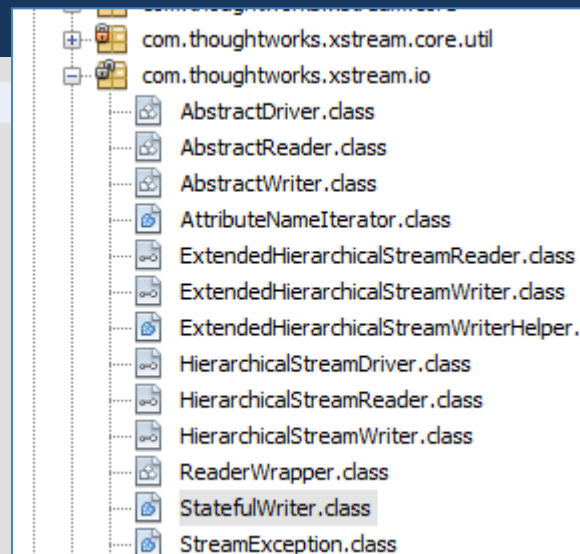
```
         * 22: putfield      com/thoughtworks/xstream/io/StatefulWriter.attributes:Lcom/t
```

```
         * 25: return
```

```
         * */
```

```
        // </editor-fold>
```

```
    }
```





3. Виртуализация ОС

История:

- 1970-е гг. в мэйнфреймах IBM System 360/370 появились полноценные виртуальные машины – **VM/370**. В операционной системе VM/370 пользователь получал в свое распоряжение полноразмерный и полнофункциональный виртуальный компьютер, на который он мог поставить собственную версию операционной системы и установить собственное прикладное программное обеспечение. Этот компьютер включал оперативную память, ресурсы процессора, собственные виртуальные периферийные устройства
- **VMWare** – это программное обеспечение для виртуальных машин (далее VM), выпущенное компанией VMWare Inc. в Пало-Альто (штат Калифорния). Компания предоставляет различное ПО и приложения для виртуализации. Она стала одним из ключевых поставщиков данной продукции. Программное обеспечение VMWare Workstation было запущено в мае 1999 года.



Виртуализация ОС. Hypervisor

Гипервизор (Hypervisor) – прослойка между «железом» и кодом *гостевой ОС*:

- **Virtual Machine Monitor (VMM)** (Монитор Виртуальной Машины): Используется для отделения и эмуляции набора привилегированных инструкций (которые может исполнять лишь само ядро его операционной системы)
- **Device model** (Модель Устройства): Используется для виртуализации имеющихся устройств ввода/ вывода

Требования к гипервизору: (Popek and Goldberg, 1973):

1. **Изоляция**: Обязан изолировать *гостевые ОС* (VM) друг от друга
2. **Эквивалентности**: Должен вести себя точно так же, как при виртуализации, так и без неё. Это означает, что большинство (почти все) инструкций исполняется в «железе» без какой бы то ни было трансляции
3. **Производительность**: Сопоставимость производительности *гостевой ОС* под VM и без нее. Минимальные издержки запуска VM.

<https://habr.com/ru/articles/474776/> - виды виртуализации и архитектур гипервизоров



Виртуализация ОС. Типы гипервизоров

Тип 1. native, bare-metal - автономный, тонкий, исполняемый на «голом железе», фактически специализированная ОС. **VMware ESX / ESXi, Citrix XenServer.**

Тип 2. Хостовый, монитор виртуальных машин — **hosted, Type 2, V** — специальный дополнительный программный слой, расположенный поверх основной хостовой ОС, который в основном выполняет функции управления гостевыми ОС, а эмуляцию и управление аппаратурой берет на себя хостовая ОС. Гипервизор - работает как приложение , гостевая ОС выполняется как процесс на хосте, а гипервизор разделяет гостевую ОС и ОС хоста. **VMware Workstation, VMware Player**

Тип 1.5, 1+ - Гибридный , функции управления аппаратными средствами выполняются тонким гипервизором и специальной депривилегированной (корневой) сервисной ОС, работающей под управлением тонкого гипервизора. Гипервизор управляет напрямую процессором и памятью компьютера, а через сервисную ОС гостевые ОС работают с остальными аппаратными компонентами. Через сервисную ОС гостевые ОС получают доступ к физическому оборудованию. Примерами данного вида гипервизоров выступают: **Microsoft Virtual Server, Sun Logical Domains, Xen, Citrix XenServer, Microsoft Hyper-V, VMware Workstation**



Виртуализация ОС. Hypervisor

Гипервизор (Hypervisor) – прослойка между «железом» и кодом *гостевой ОС*:

- **Virtual Machine Monitor (VMM)** (Монитор Виртуальной Машины): Используется для отделения и эмуляции набора привилегированных инструкций (которые может исполнять лишь само ядро его операционной системы)
- **Device model** (Модель Устройства): Используется для виртуализации имеющихся устройств ввода/ вывода

Требования к гипервизору: (Popek and Goldberg, 1973):

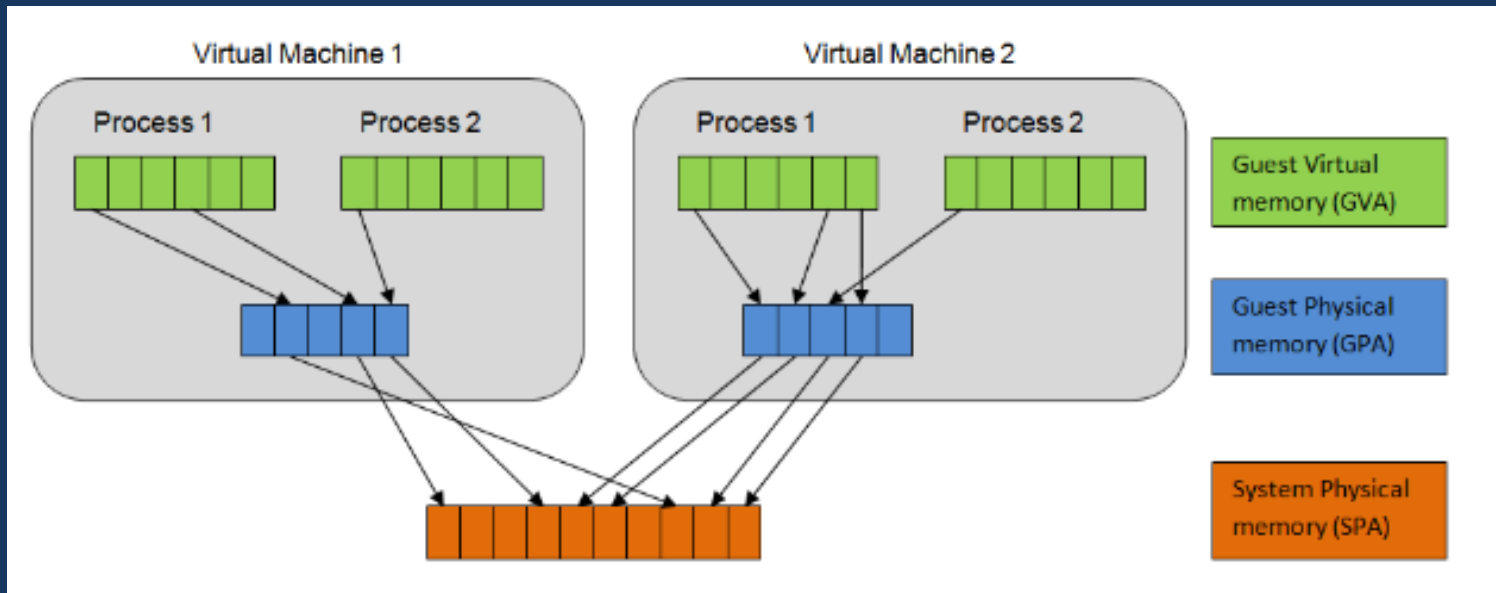
1. **Изоляция**: Обязан изолировать *гостевые ОС* (ВМ) друг от друга
2. **Эквивалентности**: Должен вести себя точно так же, как при виртуализации, так и без неё. Это означает, что большинство (почти все) инструкций исполняется в «железе» без какой бы то ни было трансляции
3. **Производительность**: Сопоставимость производительности *гостевой ОС* под ВМ и без нее. Минимальные издержки запуска ВМ.



Hyperervisor. Виртуализация памяти

Адресное пространство систем ВМ:

1. **Гостевая виртуальная память:** уровень виртуализации АП, аналогичный ВАП обычной ОС (уровень ВАП-2).
GVA - Guest Virtual Address
2. **Гостевая физическая память:** ФАП в *гостевой ОС* (уровень ВАП-1)
GPA - Guest Physical Address
3. **Системная физическая память:** ФАП - АП физических адресов процессора.
SPA - System Physical Address



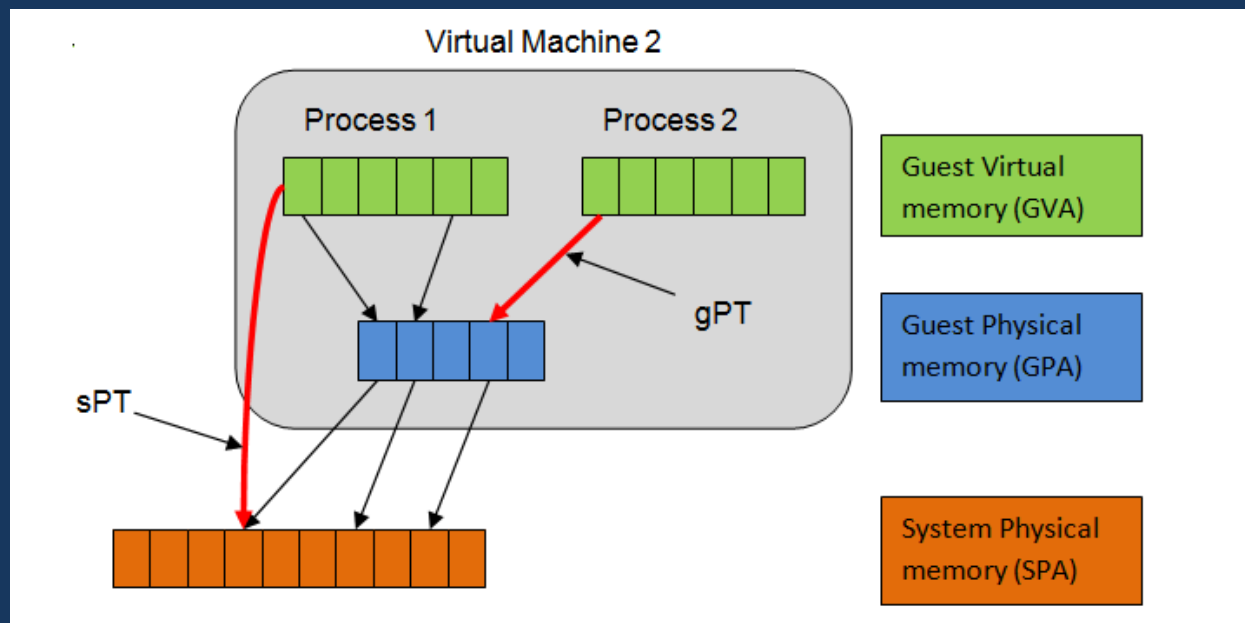
<https://www.tune-it.ru/en/web/il/home/-/blogs/виртуальная-виртуальная-память-1-shadow-tables>



Hyperervisor. Виртуализация памяти

Вариант 1: *тенивая таблица страниц (Shadow PagesTable)*:

- таблица преобразования GVA – SPA, управляемая гипервизором. Гипервизор «подменяет» CR3, установленную *гостевой ОС*, на адрес теневых таблиц страниц
- отслеживание изменений, вносимых гостевой ОС в собственную таблицу страниц Guest Pag Table
 - Guest Pag Table размещается в ВАП, для ее страниц установлен Cache Disabled — кэширование запрещено, любая запись вызывает страничное прерывание (сохранение измененных данных)



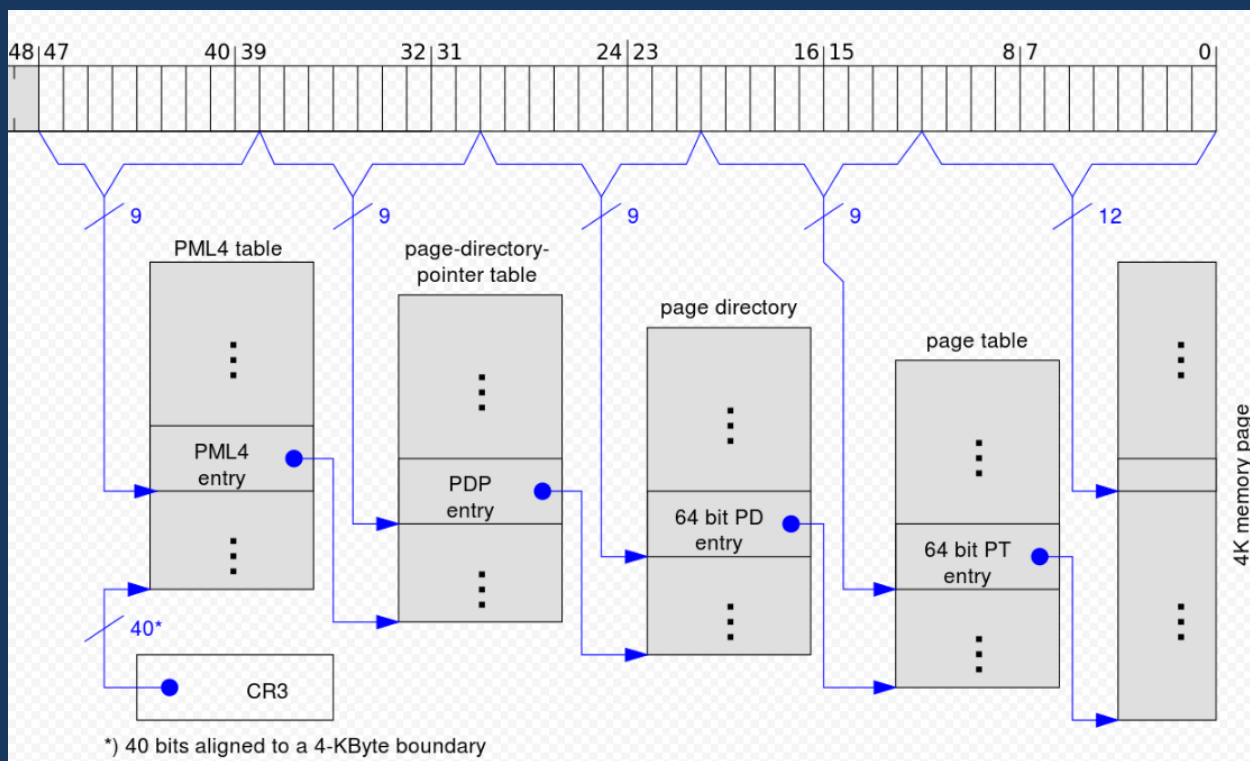


Hyperervisor. Виртуализация памяти

Вариант 2: **Вложенные таблицы страниц с аппаратной поддержкой**

Intel и AMD предоставляют решение через аппаратные расширения. Intel - *EPT* (*Extended Page Table*, Расширенная таблица страниц), что делает возможным для MMU проходить по двум таблицам страниц

- 4 уровня таблиц преобразования GVA – GPA
- 4 уровня таблиц преобразования GPA – SPA (EPT)





Hypervisor. Виртуализация процессора

Кольца защиты: аппаратный уровень процессора (0-3), определяющий возможности выполнения тех или иных привилегированных команд (или прерываний при их исполнении).

Полная виртуализация. Соответствующие инструкции отлавливаются и эмулируются для надлежащей целевой среды. Это вызывает большое число накладных расходов, поскольку должно перехватываться большое число инструкций в соответствующем хосте/ гипервизоре с последующей эмуляцией

Паравиртуализация. *Гостевая ОС* знает, что она в виртуальной машине, и знает как можно обратиться к хостовой ОС за определенными системными функциями. Таким образом снимается проблема эмуляции нулевого кольца — гостевая ОС знает, что она не в нулевом и ведет себя соответственно.



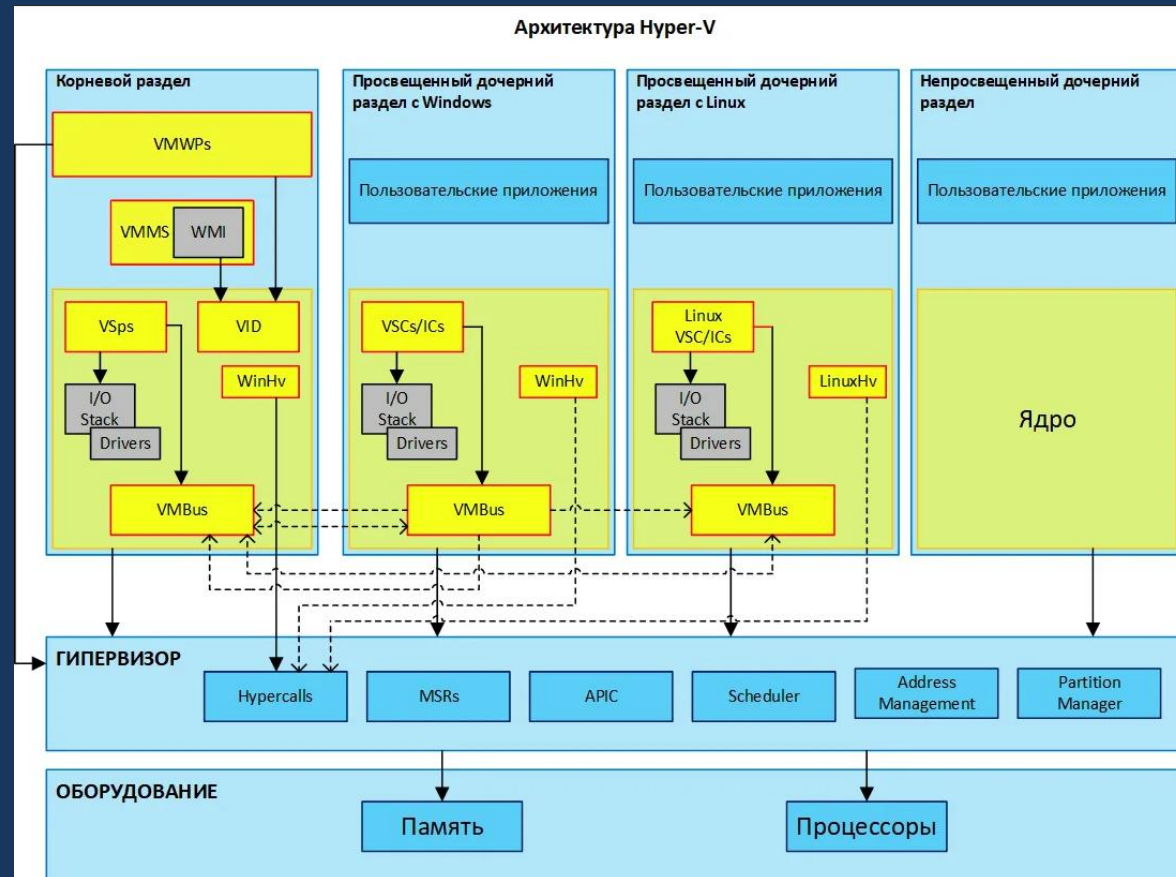
Hypervisor. Виртуализация ввода/вывода

Модели виртуализации ввода/ вывода:

- **Полная виртуализация.** Гостевая ОС не осведомлена о том, что она запущена в некотором гипервизоре и не нуждается ни в каких изменениях для запуска в гипервизоре. Всякий раз, когда такой гостевая ОС выполняет вызовы ввода/вывода, они перехватываются в гипервизоре и сам гипервизор выполняет этот ввод/ вывод в соответствующем устройстве

- **Паравиртуализация.**

Гостевая ОС осведомлена о том, что она запущена в виртуальной среде и использует специальные драйверы. Соответствующие системные вызовы для ввода/ вывода заменяются гипервызовами.





Hypervisor. Виртуализация ввода/вывода

Hyper-V - **раздел** — логическая единица разграничения, поддерживаемая гипервизором, в котором работают ОС. Каждый экземпляр гипервизора должен иметь один **родительский (корневой) раздел** с запущенной Windows Server 2008. Стек виртуализации запускается на родительском разделе и обладает прямым доступом к аппаратным устройствам. Родительский раздел порождает *дочерние разделы*, на которых и располагаются гостевые ОС. Родительский раздел создает дочерние при помощи *API-гипервизора*.

Виртуализированные разделы не имеют ни доступа к физическому процессору, ни возможности управлять его реальными прерываниями. Вместо этого у них есть виртуальное представление процессора и гостевое АП.

Гипервизор управляет прерываниями процессора и перенаправляет их в соответствующий раздел, используя логический *контроллер искусственных прерываний* (*Synthetic Interrupt Controller* или сокр. SynIC).

Дочерние разделы получают виртуальное представление ресурсов, называемое *виртуальными устройствами*. Любая попытка обращения к виртуальным устройствам перенаправляется через *VMBus* к устройствам родительского раздела, которые и обработают данный запрос.



Hyperervisor. Виртуализация ввода/вывода

VMBus — это логический канал, осуществляющий взаимодействие между разделами. Ответ возвращается также через VMBus. Если устройства родительского раздела также являются виртуальными устройствами, то запрос будет передаваться дальше, пока не достигнет такого родительского раздела, где он получит доступ к физическим устройствам. Родительские разделы запускают *провайдер сервиса виртуализации (Virtualization Service Provider* или сокр. VSP), который соединяется с VMBus и обрабатывает запросы доступа к устройствам от дочерних разделов. Виртуальные устройства дочернего раздела работают с *клиентом сервиса виртуализации (Virtualization Service Client ,VSC)*, который перенаправляет запрос через VMBus к VSP родительского раздела. Этот процесс прозрачен для гостевой ОС.



WSL - Windows-Subsystem-for-Linux

«So like DOS before him, this Windows must die»

Реминисценция на «Jesus Christ Superstar»

<http://onreader.mdl.ru/LearnWindowsSubsystemLinuxProfessionals/content/index.html>

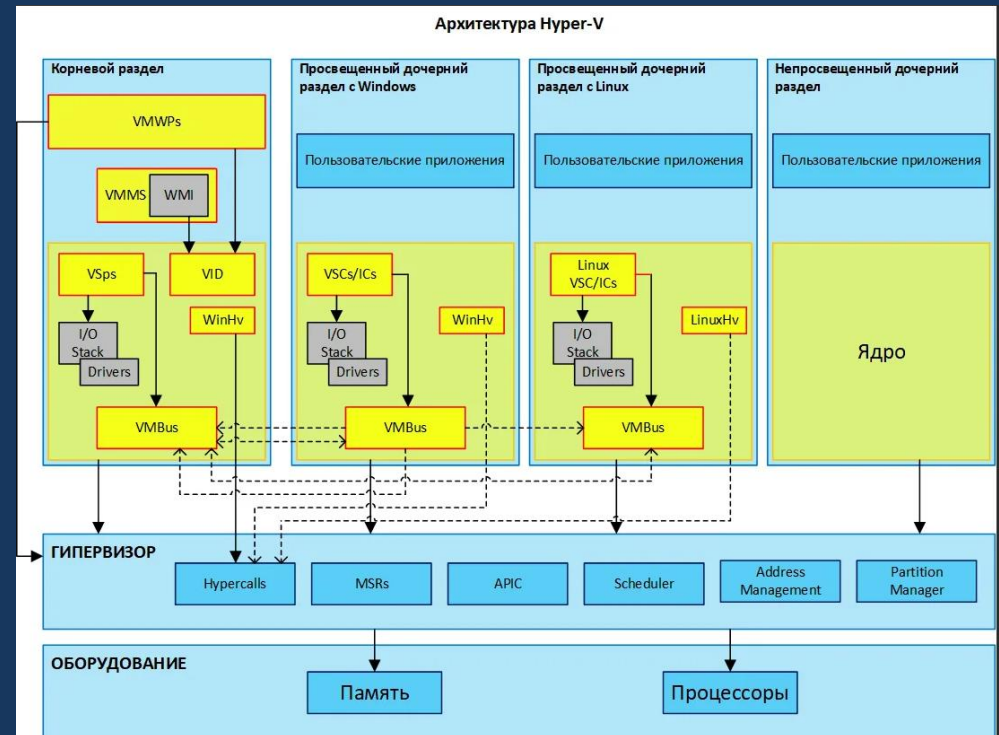
Изучаем подсистемы Windows для Linux

<https://techrocks.ru/2019/11/17/windows-subsystem-for-linux-wsl/> - популярно

WSL - Windows-Subsystem-for-Linux:

- WSL1 - подсистема позволяет в Windows запускать программы GNU/Linux в двоичном исполняемом формате ELF-64 (эмуляция API процесса)

- WSL2 — виртуальная машина с Linux на основе архитектуры Hyper-V (гипервизор, аппаратно-программная изоляция среды ОС от «железа», создание виртуальной среды исполнения кода ОС (в т.ч. ядра)





WSL – запуск

PowerShell - расширяемое средство автоматизации от Microsoft с открытым исходным кодом, состоящее из оболочки с **интерфейсом командной строки** и сопутствующего языка сценариев

The screenshot illustrates the process of launching Windows PowerShell from a file explorer window. In the file explorer, the 'temp' folder is selected, and the 'Run Windows PowerShell' option is highlighted in the context menu. The PowerShell window then displays the command `PS F:\temp> dir` and the resulting directory listing.

Каталог: F:\temp

Mode	LastWriteTime	Length	Name
d----	02.08.2022 17:51		.vscode
d----	15.04.2022 12:33		fufu
d----	12.04.2022 17:26		solus_rex-brs_android-64563b0d6756
d----	08.06.2022 15:15		SparkRetrofitExample
d----	23.06.2022 21:54		tooloud
d----	02.08.2022 18:26		WSLGlut
d----	18.07.2022 13:39		ЛР Моделирование ВП (Java)-2
d----	15.07.2022 20:28		Луч солнца золотого 2
-a----	02.08.2022 18:10	16832	44-06
-a----	02.08.2022 16:58	703	44-06.cpp
-a----	02.08.2022 16:35	23712	46-16
-a----	16.07.2022 12:44	1607	46-16.cpp
-a----	15.07.2022 21:33	17040	46-16.exe1
-a----	15.07.2022 22:45	3360	46-16.o
-a----	02.08.2022 16:39	16824	a.out
-a----	15.07.2022 21:17	18	fufu.cpp

WSL запускается как команда в PowerShell

<https://docs.microsoft.com/ru-ru/windows/wsl/basic-commands>



WSL – команды

Просмотр доступных и установленных дистрибутивов Linux

```
PS F:\temp> wsl --list --verbose
  NAME      STATE      VERSION
* Ubuntu    Stopped    2
PS F:\temp> wsl --list --online
```

Ниже приведен список допустимых распределений, которые можно установить.
Установите с помощью команды `wsl --install -d <Distro>`.

NAME	FRIENDLY NAME
Ubuntu	Ubuntu
Debian	Debian GNU/Linux
kali-linux	Kali Linux Rolling
opensUSE-42	opensUSE Leap 42
SLES-12	SUSE Linux Enterprise Server v12
Ubuntu-16.04	Ubuntu 16.04 LTS
Ubuntu-18.04	Ubuntu 18.04 LTS
Ubuntu-20.04	Ubuntu 20.04 LTS

```
PS F:\temp>
```

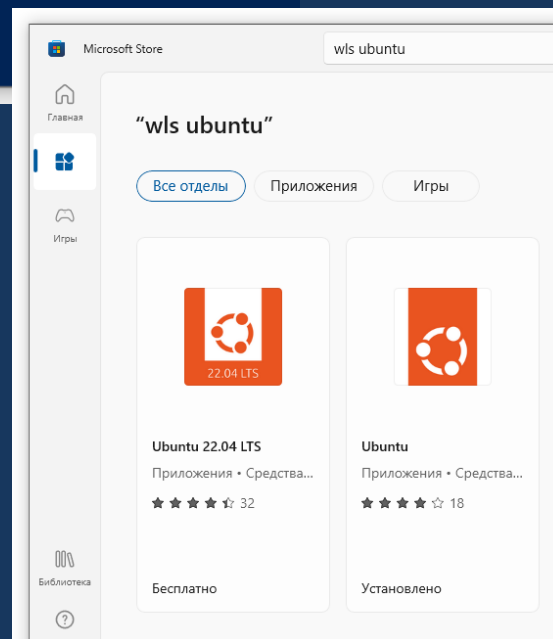
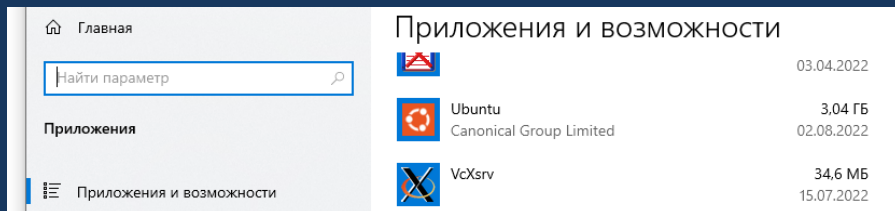
Установка дистрибутива из списка

Примеры:

```
wsl --install -d Ubuntu
wsl --install --distribution Debian
```

Установка дистрибутива online

<https://aka.ms/wslstore> - wsl Ubuntu



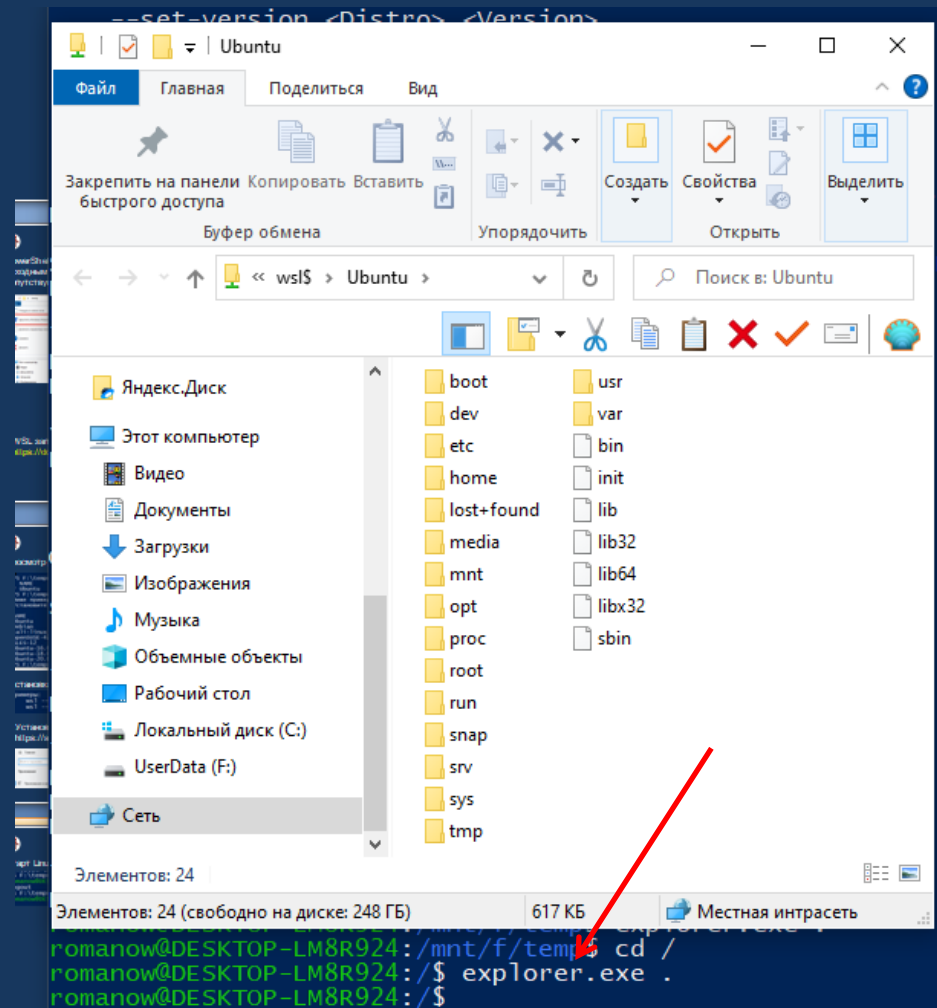
При установке – логин/пароль



WSL – команды

Старт Linux и авторизация

```
PS F:\temp> wsl -u romanow
romanow@DESKTOP-LM8R924:/mnt/f/temp$ exit
logout
PS F:\temp> wsl -d ubuntu
romanow@DESKTOP-LM8R924:/mnt/f/temp$
```



Смешанное исполнение программ в Linux и Windows, проводник Windows с корневым каталогом Linux



WSL /Ubuntu – установка компонент

Установка c/c++ в wsl/Ubuntu

```
sudo apt-get update  
sudo apt install gcc  
sudo apt install g++  
sudo apt-get install build-essential gdb
```

Установка CodeBlocks

```
sudo add-apt-repository ppa:damien-moore/codeblocks-stable  
sudo apt update  
sudo apt install codeblocks codeblocks-contrib
```

Установка freeGlut (3D-графика OpenGL для РГР)

```
sudo apt-get install freeglut3 freeglut3-dev  
sudo apt-get install binutils-gold
```

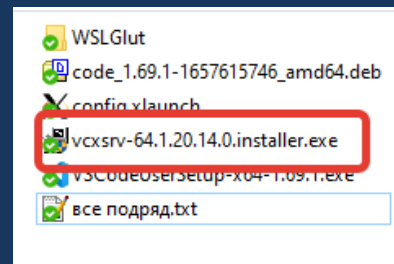



Графика для приложений Linux в WSL

VcXsrv под Windows - эмуляция граф. интерфейса Linux под Windows – сервер графики
<https://sourceforge.net/projects/vcxsrv/>

При запуске XLaunch в 3 окне по счету "Extra settings"

- Disable Access Control
- в поле "Additional parameters for VcXsrv" ввести строку:
-xkblayout us,ru -xkbvariant winkeys -xkboptions grp:ctrl_shift_toggle



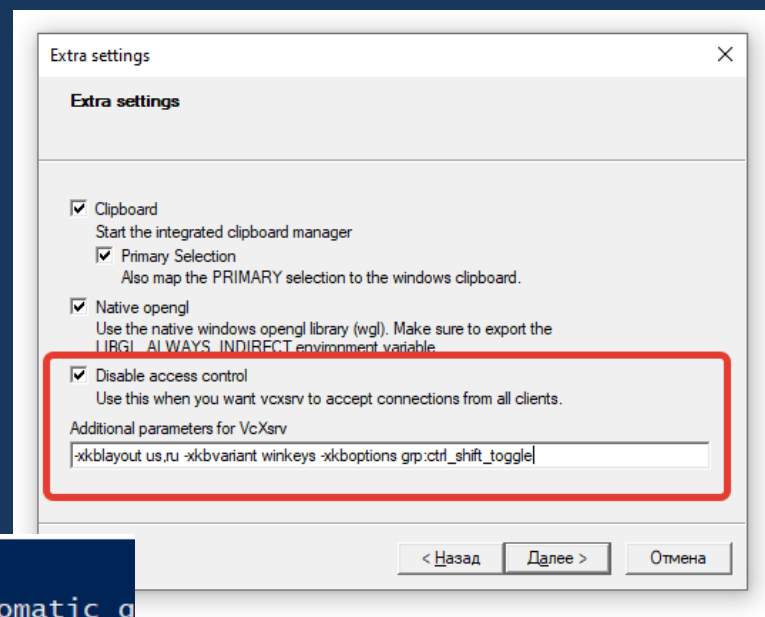
Linux (WSL):

cat /etc/resolv.conf

- получить ip, через который работать

172.31.224.1, выполнить команду

export DISPLAY=172.31.224.1:0

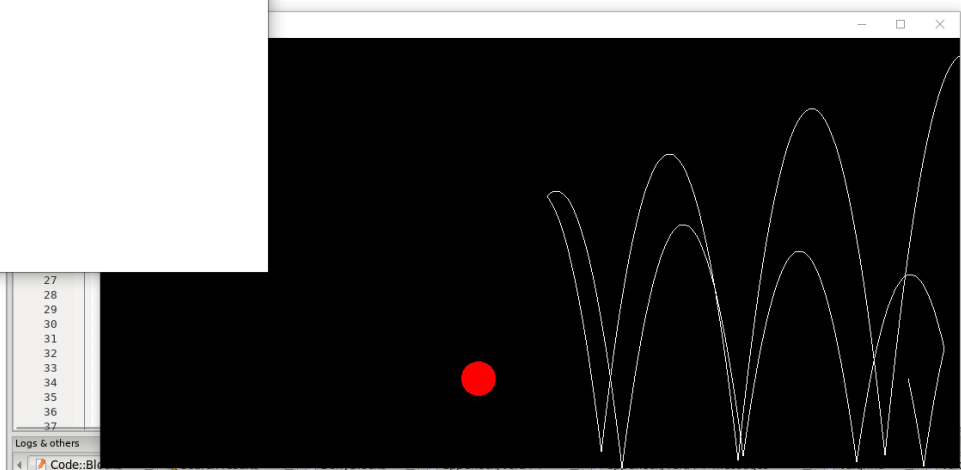
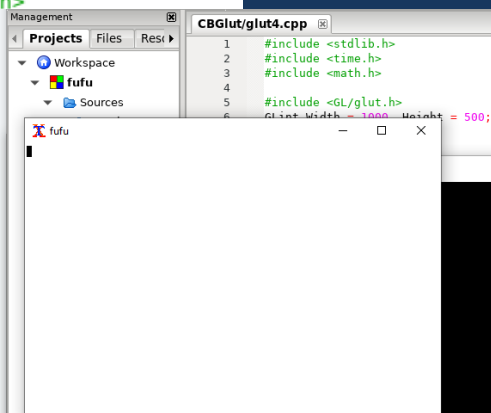
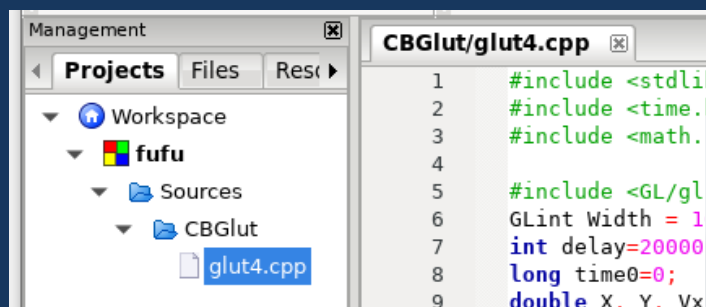
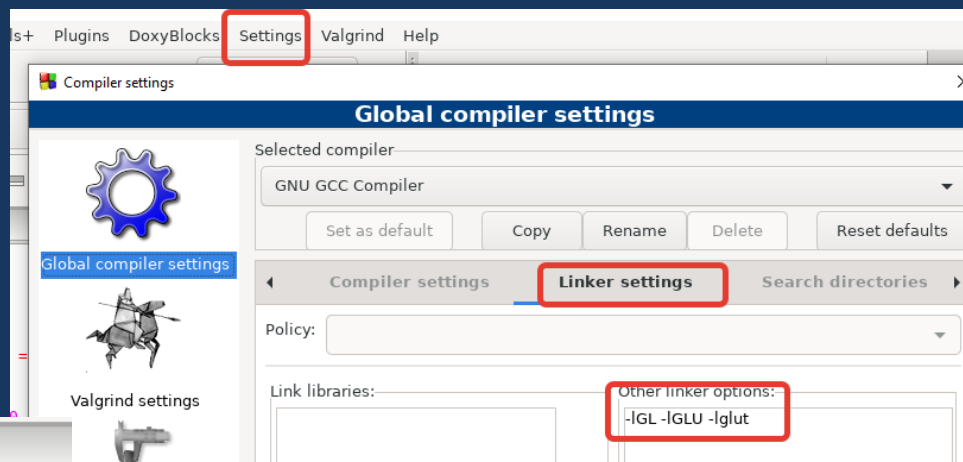


```
PS F:\temp> wsl -u romanow
romanow@DESKTOP-LM8R924:/mnt/f/temp$ cat /etc/resolv.conf
# This file was automatically generated by WSL. To stop automatic g
tc/wsl.conf:
# [network]
# generateResolvConf = false
nameserver 172.31.224.1
romanow@DESKTOP-LM8R924:/mnt/f/temp$ export DISPLAY=172.31.224.1:0
romanow@DESKTOP-LM8R924:/mnt/f/temp$
```



WSL CodeBlocks/FreeGlut

1. Запустить и настроить VcXsrv под Windows, настроить в WSL (см.выше)
2. Запустить CodeBlocks под WSL
3. Меню Settings – Compiler – Linker Settings – Поле Other Linker Options -IGL -IGLU -lglut
1. Создать проект Console Application
2. Добавить файл примера





4. Виртуализация. Контейнеры. Docker

*Иа-Иа, увидев **контейнер**, очень оживился.
— Вот это да! — закричал он. — Знаете что? Мой шарик как
раз войдет в этот **контейнер**!*

*— Что ты, что ты, Иа, — сказал Пух. — Воздушные шары не
входят в **контейнеры**. Они слишком большие. Ты с ними не
умеешь обращаться. Нужно вот как: возьми шарик за вере...*

...

*— Мне очень приятно, — радостно сказал Пух, — что я
догадался подарить тебе **Полезный Контейнер**, куда можно
класть какие хочешь **Приложения**!*

*— А мне очень приятно, — радостно сказал Пятачок, — что я
догадался подарить тебе такое **Приложение**, которое можно
класть в этот **Полезный Контейнер**!*

*Но Иа-Иа ничего не слышал. Ему было не до того: он, то клал
свой шар в **контейнер**, то вынимал его обратно, и видно было,
что он совершенно счастлив!*

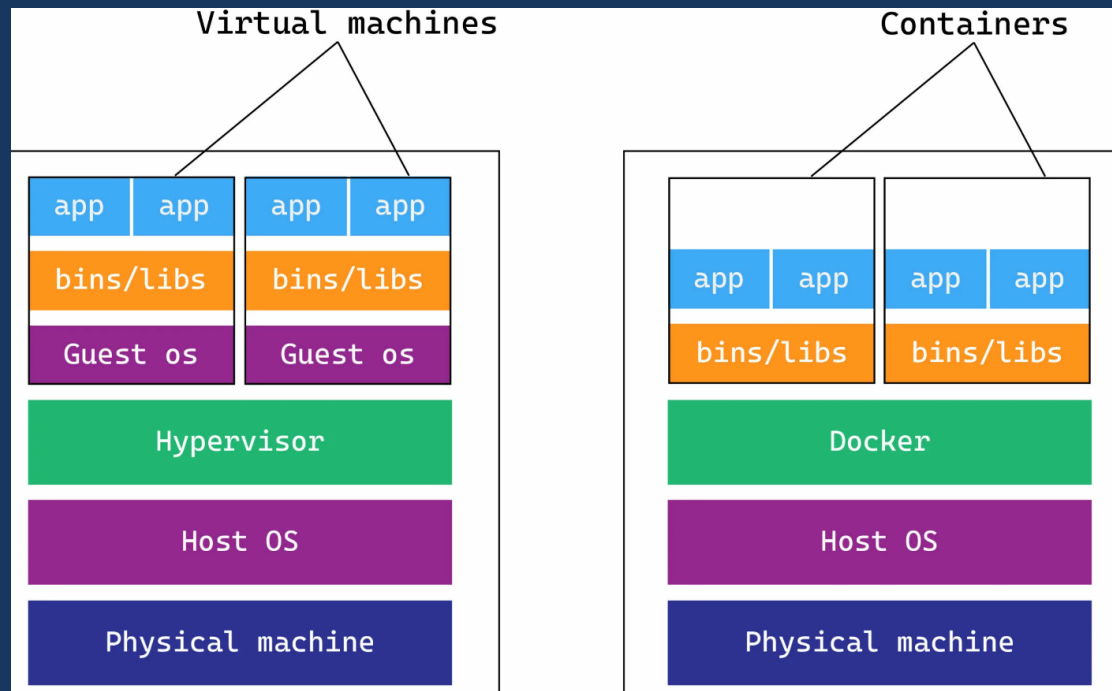
Вариации из Винни Пуха на тему Docker



Контейнеризация. Docker

Контейнеризация: изоляция приложения и его окружения (зависимостей):

- изоляция приложений
- собственное окружение (зависимости)
- быстрое развертывание
- платформенная зависимость: контейнер «заточен» под определенную ОС (Windows: WSL)
- микросервисная архитектура: взаимодействующие контейнеры
- альтернатива виртуальным машинам





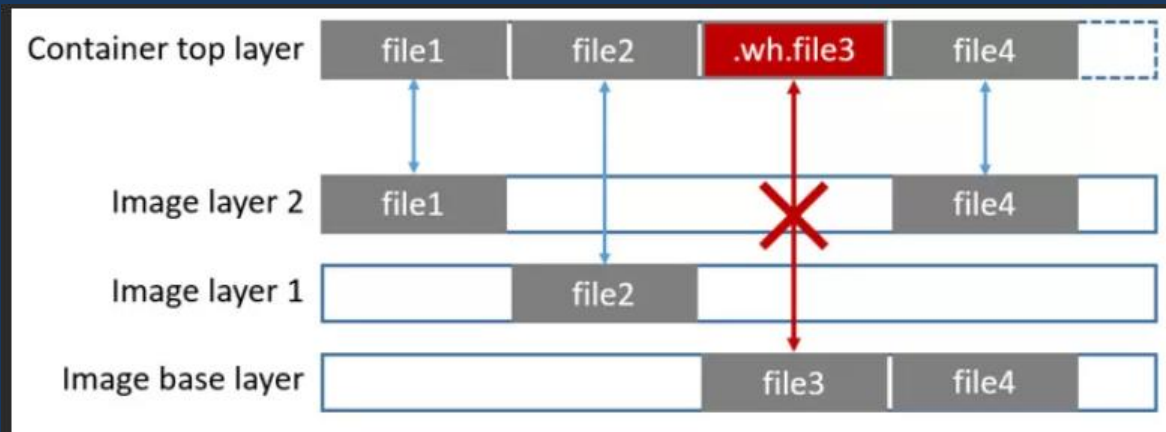
Контейнеризация. Docker

Технологии изоляции приложений:

- «собственная» файловая система
- внешнее управление, параметры окружения (пространство имен)

Технология - каскадно объединённая файловая система:

- создается монтируется виртуальный каталог (слой) как путем отображения на несколько каталогов (слоев) - merged
- верхний слой – read/write
- следующие – read only
- удаление файла «логическое»



UnionFS , OverlayFS – надстройка (проxy) над базовой ФС

<https://habr.com/ru/companies/skillfactory/articles/547116/>



Контейнеры. Изоляция

Предпосылки изоляции:

- клонирование образа из репозитория (*родительский*). Слои данных нового образа накладываются поверх слоев родительского
- DockerHub – репозиторий образов. 51% образов содержит критические уязвимости. Все уязвимости родительского образа наследуются дочерними.

1. **Модули безопасности Linux** (Linux Security Modules, LSM) позволяют реализовать нестандартные для Linux-систем модели безопасности. Механизм мандатного разграничения доступа (англ. Mandatory Access Control, MAC): SELinux, AppArmor – метки конфиденциальности и уровни доступа.
2. **Контрольные группы** (control groups, сокр. cgroups) — функция ядра Linux, позволяющий контролировать потребление системных ресурсов процессами. Ограничение потребления ресурсов для контрольных групп достигается путем встраивания их в *контроллеры* (resource controllers), которые также называются *подсистемами* (subsystem). Управление cgroups производится через виртуальную файловую систему. Контрольные группы для разных контроллеров организованы в иерархии, дочерние группы наследуют атрибуты родительских.

<https://www.okbsapr.ru/library/publications/programmnye-mekhanizmy-izolyatsii-konteynerov-docker/>

<https://habr.com/ru/companies/selectel/articles/303190/>



Контейнеры. Изоляция. Control groups

Контроллеры:

- blkio** — устанавливает лимиты на чтение и запись с блочных устройств
- cpuacct** — генерирует отчёты об использовании ресурсов процессора
- cpu** — обеспечивает доступ процессов в рамках контрольной группы к CPU
- cpuset** — распределяет задачи в рамках контрольной группы между процессорными ядрами
- devices** — разрешает или блокирует доступ к устройствам
- freezer** — приостанавливает и возобновляет выполнение задач в рамках контрольной группы
- hugetlb** — активирует поддержку больших страниц памяти для контрольных групп;
- memory** — управляет выделением памяти для групп процессов
- net_cls** — помечает сетевые пакеты специальным тэгом, что позволяет идентифицировать пакеты, порождаемые определённой задачей в рамках контрольной группы
- netprio** — используется для динамической установки приоритетов по трафику;
- pids** — используется для ограничения количества процессов в рамках контрольной группы

```
romanow@DESKTOP-LM8R924:/mnt/fs$ ls /sys/fs/cgroup/  
blkio  cpu  cpuacct  cpuset  devices  freezer  hugetlb  memory  net_cls  net_pr  
io  perf_event  pids  rdma  unified  
romanow@DESKTOP-LM8R924:/mnt/fs$
```



Контейнеры. Изоляция. Control groups

Каждая подсистема представляет собой директорию с управляющими файлами, в которых прописываются все настройки. В каждой из этих директорий имеются управляющие файлы: `tasks` — содержит список PID всех процессов, включённых в контрольные группы, `cgroup.procs` — содержит список TGID групп процессов, включённых в контрольные группы

```
romanow@DESKTOP-LM8R924:/mnt/f$ ls /sys/fs/cgroup/
blkio  cpu  cpuacct  cpuset  devices  freezer  hugetlb  memory  net_cls  net_pr
io  perf_event  pids  rdma  unified
romanow@DESKTOP-LM8R924:/mnt/f$ ls /sys/fs/cgroup/cpu
cgroup.clone_children  cpu.cfs_period_us  cpu.rt_runtime_us  notify_on_release
cgroup.procs           cpu.cfs_quota_us  cpu.shares         release_agent
cgroup.sane_behavior   cpu.rt_period_us  cpu.stat           tasks
romanow@DESKTOP-LM8R924:/mnt/f$ cat /sys/fs/cgroup/cpu/cgroup.procs
1
8
9
10
75
romanow@DESKTOP-LM8R924:/mnt/f$ cat /sys/fs/cgroup/cpu/tasks
1
7
8
9
10
76
```

Чтобы создать контрольную группу, достаточно создать вложенную директорию в любой из подсистем. В эту вложенную директорию будут автоматически добавлены управляющие файлы. Добавить процессы в группу: записать их PID в управляющий файл `tasks`



Контейнеры. Изоляция. Control groups 2

Control groups V2. Для создания группы создать папку и смонтировать на ней ФС cgroups

```
romanow@DESKTOP-LM8R924:/mnt/f/WSL$ mkdir mygroup
romanow@DESKTOP-LM8R924:/mnt/f/WSL$ mount -t cgroup2 none mygroup
mount: only root can use "--types" option
romanow@DESKTOP-LM8R924:/mnt/f/WSL$ sudo mount -t cgroup2 none mygroup
romanow@DESKTOP-LM8R924:/mnt/f/WSL$ ls mygroup
cgroup.controllers  cgroup.max.descendants  cgroup.stat          cgroup.threads
cgroup.max.depth    cgroup.procs           cgroup.subtree_control  cpu.stat
romanow@DESKTOP-LM8R924:/mnt/f/WSL$ _
```



Контейнеры. Пространства имен

Пространства имён (namespaces) — механизм ядра Linux, позволяющий изолировать ресурсы ОС для экземпляров процессов. Помещая процесс в пространство имен, можно ограничить ресурсы, видимые данному процессу.

При старте ОС создается пространство имен каждого типа, они называются инициализирующими (initial) или корневыми (root). Далее в них можно создавать дочерние пространства имен и помещать в них процессы. Процесс всегда находится ровно в одном пространстве имен каждого типа.

Типы пространств и изолируемых данных:

Cgroup - данные контрольных групп процессов

IPC - ресурсы межпроцессного взаимодействия, объекты IPC и очереди сообщений
POSIX

Network - набор сетевых ресурсов: список IP-адресов, список сокетов, таблица IP-маршрутизации, брандмауэр. Каждый физический или виртуальный сетевой интерфейс может находиться только в одном пространстве имен Network

Mount - точки монтирования файловой системы

PID - независимый от других пространств набор идентификаторов процессов

Time — собственное системное время

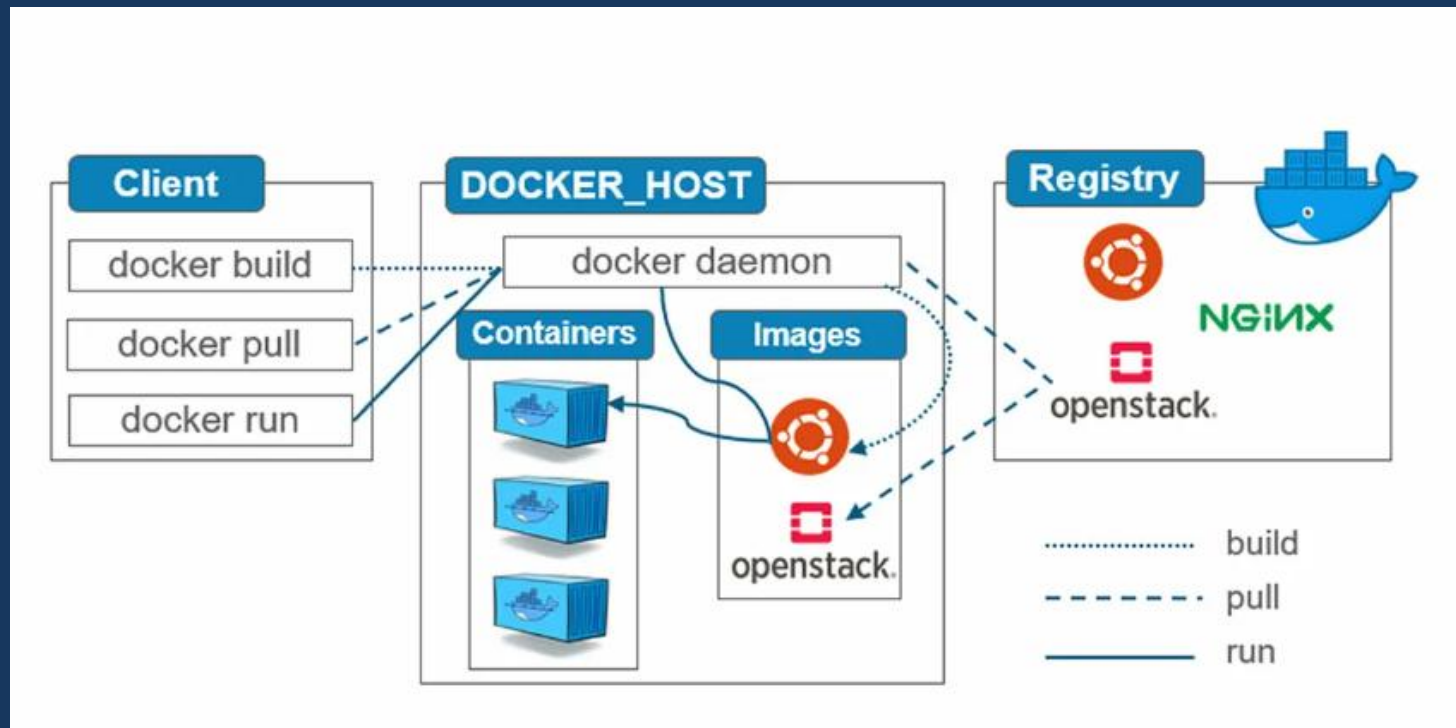
User - изоляция идентификаторов пользователей и групп, корневой каталог, ключи и привилегии

UTS - изоляция хостовых и доменных имен



Docker. Архитектура

- **Daemon** - резидентный процесс/сервер
- Клиент
- **Образ** (image) - неизменяемый файл (образ), из которого разворачиваются контейнеры
- **Контейнер** - развернутое из образа и работающее приложение
- **Registry** - репозиторий с образами
- **Dockerfile** – командный файл инструкций для сборки образа





Docker. Команды работы с контейнерами

`docker system prune`

Удалить все неактивные контейнеры Docker:

`docker logs <container-id>`

Вывести на экран stderr и stdout от запущенных в контейнере программ, логирование контейнера Docker

`docker ps`

Индекс активных контейнеров

`docker ps -a`

Индекс всех контейнеров, включая неактивные

`docker images`

Индекс образов, установленных в системе на данный момент

`docker stop <container-id>`

Деактивировать Docker-контейнер по идентификатору

`docker rm <container-id>`

Удалить неактивный контейнер по идентификатору:

`docker rmi <image-name>`

Удалить конкретный образ по его названию

`docker exec -it mongo bash`

`> mongorestore --gzip --drop --archive=ess24567.gz`

`> exit`

Создать в контейнере mongo процесс с `bash`



Docker. Контейнеры БД и сервера

mongo

mondoDB

Ess24567.gz

```
FROM mongo:4.4
RUN mkdir /opt/mongo
COPY ess24567.gz /opt/mongo
WORKDIR /opt/mongo
CMD ["mongod", "--wiredTigerCacheSizeGB", "1"]
```

```
FROM openjdk:8
RUN mkdir /opt/dataserver
RUN mkdir /opt/dataserver/4567
COPY ESS2Desktop.jar /opt/dataserver
COPY 4567 /opt/dataserver/4567
WORKDIR /opt/dataserver
CMD ["java", "-cp",
"ESS2Desktop.jar", "romanow.abc.dataserver.ESSConsoleServer", "port:
4567", "conf:BARs", "network:mongo"]
```

ess2

Java8

ESS2Desktop.jar

4567/*.*

mongo

mondoDB

Ess24567.gz

```
docker run -d --network ess2
--name ess2 -p 4567:4567
-p 4568:4568
-e TZ=Asia/Novosibirsk ess2
```

docker network create ess2

```
docker run -d --network ess2
--name mongo -v /data/db mongo
```



```
docker exec -it mongo bash
> mongorestore --gzip --drop --archive=ess24567.gz
```

ess2

Java8

Сервер

ESS2Desktop.jar

4567/*.*

4567

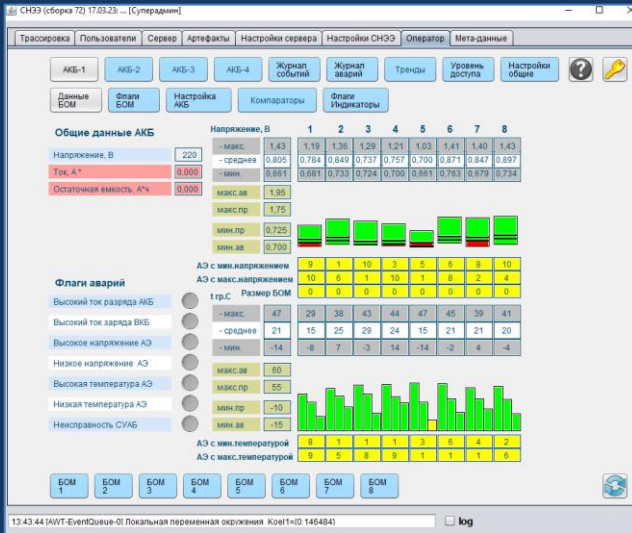
4567

ESS2Desktop.jar

Desktop-клиент



Docker. Контейнеры БД и сервера



Name	Image	Status	Port(s)	Last started	Actions
ess2 ec1ab9543ccf	ess2	Running	4567:4567	36 seconds ago	[Stop] [Restart] [Delete]
mongo be5f1491ce20	mongo	Running	27017:27017	38 seconds ago	[Stop] [Restart] [Delete]

СНЭЭ (сборка 72) 17.03.23

Сервер данных: localhost

Порт: 4567 ☒ Соединение

Логин (телефон): 9130000000

Пароль: *****

```
docker rm -f mongo ess2
docker rmi -f mongo ess2
docker network create ess2
docker build -t ess2 -f DockerfileESS2 .
docker build -t mongo -f DockerfileMongo .
docker run -d --network ess2 --name mongo -v /data/db mongo
docker exec -it mongo mongorestore --gzip --drop --archive=ess24567.gz
docker run -d --network ess2 --name ess2 -p 4567:4567 -p 4568:4568 -e TZ=Asia/Novosibirsk ess2
```



Docker. Контейнеры БД и сервера

DockerfileESS2

```
FROM openjdk:8
ENV TZ="Asia/Novosibirsk"
RUN date
RUN mkdir /opt/dataserver
RUN mkdir /opt/dataserver/4567
COPY ESS2Desktop.jar /opt/dataserver
COPY 4567 /opt/dataserver/4567
WORKDIR /opt/dataserver
CMD ["java", "-cp", "ESS2Desktop.jar", "romanow.abc.dataserver.ESSConsoleServer", "port:4567",
```

DockerfileMongo

```
FROM mongo:4.4
RUN mkdir /opt/mongo
COPY ess24567.gz /opt/mongo
WORKDIR /opt/mongo
CMD ["mongod", "--wiredTigerCacheSizeGB", "1"]
```

Инструкции Dockerfile:

FROM – установка образа контейнера из репозитория

RUN – исполнение команды Linux в процессе создания контейнера

COPY – копирование файлов (каталогов) в ФС контейнера

WORKDIR – рабочий каталог контейнера

CMD – исполнение команды в запущенном контейнере (одна !!!!)

ENV – определение переменной окружения